AI Career Agent

Comprehensive Project Documentation

Personalised Career Guidance for BTech CS Students

Stack: FastAPI + React + LangChain + Gemini AI + Firebase

Version 1.0 | July 2026

GitHub: github.com/Narwal-himanshu/AI-CAREER-AGENT

Table of Contents

1. Project Overview

AI Career Agent is an AI-powered platform built for BTech Computer Science students across India. It delivers personalised, adaptive career guidance from 1st year through 4th year - covering skill assessment, course recommendations, DSA practice, live opportunities, resume generation, and an intelligent Q&A chatbot.

The platform uses a multi-agent architecture powered by Google Gemini (via LangChain) to generate adaptive quizzes, classify skill levels, produce personalised career roadmaps, recommend courses, fetch live opportunities, and generate ATS-friendly resumes.

Core Features

- Skill Assessment Quiz - 5-question adaptive MCQ covering DSA, logic, and coding
- Skill Classification - Beginner / Intermediate / Advanced level determination
- Career Roadmap - Year-wise, domain-specific preparation timeline via LLM
- DSA Practice Sheet - 112 curated LeetCode problems across 16+ topics
- Course Recommendations - Curated courses from YouTube, Coursera, Udemy
- Live Opportunities - Hackathons, CTFs, contests from Devpost, Clist, Serper
- Resume Builder - ATS-optimised resume generation (backend ready)
- Dashboard - Personalised view with scoring breakdown, risk analysis, gaps
- Chatbot Widget - Context-aware Q&A for career guidance
- Profile Management - Firebase-backed user profiles with streak tracking

2. Tech Stack & Architecture

Frontend

- React 18: UI framework with functional components and hooks
- Vite: Build tool and dev server
- Tailwind CSS: Utility-first CSS framework for styling
- React Router v6: Client-side routing with nested routes
- Firebase Auth: Google + Email/Password authentication
- Firestore: NoSQL database for user profiles and quiz results
- Lucide React: Icon library

Backend

- FastAPI: Async Python web framework for REST API
- SQLite: Lightweight database (career_agent.db)
- LangChain: LLM orchestration framework
- Google Gemini API: LLM provider (gemini-2.5-flash / 2.0-flash)
- Pydantic v2: Data validation and schema enforcement
- Firebase Admin SDK: Server-side token verification
- Tenacity: Retry logic with exponential backoff
- Feedparser: RSS feed parsing for hackathons
- Httpx: Async HTTP client for external APIs

Architecture Diagram (Simplified)

```
Browser (React) --HTTPS--> FastAPI Backend --LangChain--> Gemini AI
                                    |                          |
                               SQLite DB                  External APIs
                            (students,                (Devpost, Clist,
                             skill_profiles)           Serper, YouTube)
                                    |
                               Firebase Auth
                            (Google + Email)
```

3. Project Structure

```
AI-CAREER-AGENT/
|
|-- backend/                      # FastAPI backend
|   |-- main.py                   # App entry point, auth, core routes
|   |-- config.py                 # Environment config (API keys, ports)
|   |-- database.py               # SQLite schema init + connection
|   |-- models/                   # Pydantic data models
|   |   |-- student.py            # StudentOnboarding schema
|   |   |-- quiz.py               # QuizResponse, QuizSubmission
|   |   |-- scoring.py            # SkillProfileOutput schema
|   |   |-- summary.py            # SummaryOutput schema
|   |   |-- risk.py               # RiskOutput schema
|   |   |-- schemas.py            # Agent request/response schemas
|   |-- agents/                   # AI Agent implementations
|   |   |-- combined_agent.py     # Core agent (quiz, score, summary, risk)
|   |   |-- career_roadmap_agent.py
|   |   |-- dsa_agent.py          # DSA sheet + LeetCode fetcher
|   |   |-- course_agent.py       # Course recommendations
|   |   |-- opportunities_agent.py # Live hackathons/contests
|   |-- services/
|   |   |-- agent_service.py      # Orchestrator service layer
|   |-- routes/
|   |   |-- agents.py             # /api/agents/* endpoints
|   |-- career_agent.db           # SQLite database file
|
|-- frontend/                     # React frontend (Vite)
|   |-- src/
|   |   |-- App.jsx               # Root component, routing, auth
|   |   |-- firebase.js           # Firebase config
|   |   |-- pages/                # 12 page components
|   |   |-- components/           # Shared UI components
|   |   |-- data/quizBank.js      # Local quiz questions + evaluator
|   |   |-- lib/
|   |       |-- profileStore.js   # Firestore CRUD operations
|   |       |-- yearNav.js        # Year slug utilities
|
|-- README.md                     # Prompt engineering doc
|-- package-lock.json
```

4. Backend (FastAPI + Python)

4.1 API Endpoints

Method	Endpoint	Description	Auth
GET	/health	Health check	No
GET	/me/status	Onboarding + quiz progress	Yes
GET	/api/onboarding	Get saved profile	Yes
POST	/api/onboarding	Save student profile	Yes
GET	/api/quiz/generate	Generate quiz from profile	Yes
POST	/api/quiz/generate	Generate quiz with payload	Yes
POST	/api/quiz/submit	Submit + analyse quiz	Yes
GET	/api/dashboard	Get cached analysis	Yes
POST	/api/agents/career-roadmap	Generate roadmap	No*
POST	/api/agents/dsa-sheet	Generate DSA sheet	No*
GET	/api/agents/dsa-topics	List DSA topics	No
POST	/api/agents/courses	Course recommendations	No*
POST	/api/agents/opportunities	Live opportunities	No*

* Agent routes are not behind auth middleware yet (identified gap).

4.2 Database Schema (SQLite)

The backend uses SQLite with two main tables:

students table

- student_id TEXT PRIMARY KEY
- firebase_uid TEXT UNIQUE NOT NULL
- name TEXT, email TEXT
- year INTEGER, branch TEXT
- cgpa REAL, college TEXT
- college_tier TEXT
- domain_interest TEXT (JSON)
- career_goal TEXT
- hours_per_day INTEGER
- preferred_style TEXT (JSON)
- created_at TEXT

skill_profiles table

- student_id TEXT PRIMARY KEY (FK)
- overall_score INTEGER
- level TEXT (Beginner/Intermediate/Advanced)
- category_scores TEXT (JSON)
- classification_reason TEXT
- weak_areas TEXT (JSON array)
- summary_text TEXT

- focus_areas TEXT (JSON array)
- recommended_next_step TEXT
- overall_risk_level TEXT
- risk_report TEXT (JSON)
- created_at TEXT

4.3 AI Agents (Backend)

The backend implements 5 AI agents using LangChain + Google Gemini. Each agent has a system prompt, input schema, output schema, and mock fallback.

Agent	Responsibility	LLM Model
CombinedAgent	Quiz gen + Scoring + Summary + Risk	gemini-2.5-flash
CareerRoadmapAgent	Year-wise career roadmap	gemini-2.0-flash
DSAPracticeAgent	112 LeetCode problems + API fetch	gemini-2.0-flash
CourseRecommendationAgent	Curated courses + YouTube	gemini-2.0-flash
OpportunitiesAgent	Hackathons, CTFs, contests	gemini-2.0-flash

4.4 Authentication Flow

1. Frontend uses Firebase Auth (Google + Email/Password).
2. On login, frontend receives a Firebase ID token.
3. Token is sent as 'Authorization: Bearer <token>' header.
4. Backend's get_current_student() dependency verifies the token.
5. Firebase UID is resolved to a student_id via SQLite lookup/insert.
6. Mock auth mode available for development (tokens starting with 'mock-uid-').

5. Frontend (React + Vite + Tailwind)

5.1 Pages & Routes

Route	Component	Description
/	Home	Landing page with hero, features, CTA
/about	About	About page with product info
/login	Login	Firebase Google + Email auth
/onboarding	Onboarding	3-step profile setup wizard
/quiz	QuizPage	5-question timed MCQ assessment
/dashboard	Dashboard	Results: scores, summary, risk
/opportunities	Opportunities	Live hackathons/CTFs feed
/roadmap	Roadmap	AI-generated career roadmap
/dsa	DSA	112 LeetCode problems tracker
/courses	Courses	Course recommendations
/resume	Resume	Resume builder (placeholder)
/profile	Profile	User profile management

5.2 Key Components

- Sidebar - Collapsible navigation with year-wise sub-menu
- UserMenu - Avatar dropdown with profile, restart, sign-out
- ChatbotWidget - Floating AI chatbot on dashboard
- SharedLanding - AnnouncementBar, Navbar, FinalCTA, Footer
- YearDropdown - Year-wise roadmap navigation

5.3 State Management

The app uses React state (useState/useEffect) with no external state library. Key state lives in App.jsx and flows down as props:

- authUser / authToken - Firebase auth state
- userDoc - Firestore user document (profile, streak, analysis)
- studentProfile - Current quiz payload
- quizQuestions - Generated quiz questions
- quizAnalysis - Completed analysis results
- authLoading / profileLoading - Loading guards for protected routes

6. AI Agent System - Detailed

All AI agents follow the same pattern: System Prompt -> User Prompt -> Gemini API -> JSON Parse -> Pydantic Validation -> Fallback to Mock Data on failure.

6.1 QuestionAgent (combined_agent.py)

Purpose: Generates adaptive MCQs for skill assessment quiz.
Model: gemini-2.5-flash | Temperature: 0.7 | Max Tokens: 3000
Input: Student profile (year, domain, career goal)
Output: QuizResponse with 5 questions (Easy/Medium/Hard distribution)
Logic: Year-based difficulty distribution (Y1: 5E/3M/2H, Y2: 4E/4M/2H, etc.)
Domain-specific topics mapped per student's chosen domain.

6.2 ScoringAgent (combined_agent.py)

Purpose: Scores quiz and classifies skill level.
Model: gemini-2.5-flash | Temperature: 0.2
Scoring Formula:
- Topic Score = (correct/total) * 100
- Overall Score = DSA(40%) + Programming(30%) + Logic(20%) + Domain(10%)
- Difficulty Bonus: +3 pts per Hard correct (max +15)
- Time Penalty: -2 pts if avg > 90s

Classification Rules (strict):
- Beginner: score < 45 OR DSA < 30
- Intermediate: score >= 45 AND DSA >= 40
- Advanced: score >= 75 AND Med/Hard correct >= 60%

6.3 SummaryAgent (combined_agent.py)

Purpose: Synthesises student profile + quiz results into structured summary.
Temperature: 0.3 | Output: SummaryOutput schema
Produces: summary_text, skill_profile, focus_areas, placement_readiness, next_step

6.4 RiskAgent (combined_agent.py)

Purpose: Identifies skill gaps, timeline risks, strategic misalignments.
Temperature: 0.2 | Output: RiskOutput schema
Considers: Indian tech ecosystem benchmarks (FAANG, MNCs, GATE, startups)
Produces: overall_risk_level, timeline_risk, skill_gaps, quick_wins, red_flags

6.5 CareerRoadmapAgent

Purpose: Generates personalised year-by-year career roadmap.
Model: gemini-2.0-flash | Temperature: 0.4
Features: Retry with exponential backoff (3 attempts), mock fallback
Output: CareerRoadmapResponse with plan per year, resources, milestones

6.6 DSA Practice Agent

Purpose: Provides 112 curated LeetCode problems across 16 DSA topics.
Built-in Problem Set: 112 hardcoded problems (Arrays through Math/Number Theory)
Live LeetCode Fetch: GraphQL API to fetch problems by difficulty + topic
Caching: 24-hour TTL cache for API results
Topics: Arrays, Strings, Hash Table, Linked Lists, Stacks, Queues, Trees, BSTs, Graphs, Heaps, Sorting, Searching, Two Pointers, Sliding Window, Binary Search, Recursion/Backtracking, Dynamic Programming, Greedy, Bit Manipulation, Tries, Union Find, Math & Number Theory

6.7 CourseRecommendationAgent

Purpose: Recommends courses from curated list + YouTube API.
Model: gemini-2.0-flash | Temperature: 0.3
Curated courses for: AI/ML, Web Dev, DSA/CP, Cloud, CyberSec, Mobile
YouTube API integration for fresh course discovery
Caching: 12-hour TTL for recommendations

6.8 OpportunitiesAgent

Purpose: Fetches live hackathons, CTFs, contests, internships.
Sources:
1. Devpost RSS feed (hackathons)
2. Serper.dev web search (domain-specific opportunities)
3. Clist.by API (competitive programming contests)

Model: gemini-2.0-flash for parsing/filtering results
Caching: 6-hour TTL for opportunities feed

7. Data Flow & User Journey

Complete user journey through the application:

- 1. User visits landing page -> clicks 'Start Free Assessment'
- 2. Login page -> Firebase Google/Email authentication
- 3. Onboarding (3 steps):
- Step 1: Name, email, age, year, branch, college, tier, CGPA
- Step 2: Domain interests (multi-select), career goal
- Step 3: Hours/day, learning style preference
- 4. Profile saved to Firestore + local state
- 5. Quiz generated (5 MCQs from local quiz bank)
- 6. Quiz taken with per-question timer (60s countdown)
- 7. Quiz scored locally via evaluateQuizLocally()
- 8. Results saved to Firestore
- 9. Dashboard displayed with:
- - Skill level (Beginner/Intermediate/Advanced)
- - Category scores (DSA, Programming, Logic, Domain)
- - Summary analysis
- - Risk assessment with timeline
- - Quick wins and red flags
- 10. Sidebar navigation to: Roadmap, DSA, Courses, Resume, etc.
- 11. Chatbot widget available on dashboard for Q&A

Frontend vs Backend Quiz Flow

IMPORTANT: The current frontend uses a LOCAL quiz system (quizBank.js) that does NOT call the backend API. The backend quiz endpoints exist and are fully functional but the frontend bypasses them for speed/offline use. The backend agents (CareerRoadmap, DSA, Courses, Opportunities) ARE called from the frontend via /api/agents/* endpoints.

8. What Has Been Done (Implemented)

Features that are fully implemented and working:

- [DONE] Authentication: Firebase Google + Email/Password login/signup
- [DONE] Onboarding: 3-step wizard with profile, interests, goals
- [DONE] Quiz System: 5-question local MCQ quiz with timer
- [DONE] Skill Classification: Beginner/Intermediate/Advanced scoring
- [DONE] Dashboard: Full analysis view with scores, gaps, risk, timeline
- [DONE] DSA Practice: 112 LeetCode problems with topic filtering
- [DONE] Career Roadmap: AI-generated year-wise preparation plan
- [DONE] Course Recommendations: Curated + YouTube courses by domain
- [DONE] Live Opportunities: Hackathons, CTFs from 3 data sources
- [DONE] Sidebar Navigation: Collapsible with year-wise sub-menus
- [DONE] User Profile: Firestore-backed profile with streak tracking
- [DONE] Responsive Design: Mobile sidebar drawer, desktop collapse
- [DONE] Readiness Score: Animated ring on landing page
- [DONE] Chatbot Widget: Basic UI on dashboard page
- [DONE] About Page: Product information page
- [DONE] Backend API: 13 endpoints with auth, validation, error handling
- [DONE] Database: SQLite with students + skill_profiles tables
- [DONE] Agent Fallback: Mock data for all agents when API fails
- [DONE] Error Handling: Structured error codes + retry logic
- [DONE] Caching: In-memory TTL caches for DSA, courses, opportunities

9. What Can Be Done (Future Scope)

Identified gaps and improvement opportunities:

High Priority

- [TODO] Connect frontend quiz to backend API (currently uses local evaluator)
- [TODO] Add auth middleware to /api/agents/* routes (currently unprotected)
- [TODO] Implement Resume Builder (backend agent exists, frontend is placeholder)
- [TODO] Add ChatbotAgent backend integration (currently frontend-only UI)
- [TODO] Switch from SQLite to PostgreSQL for production deployment
- [TODO] Add Docker + CI/CD pipeline (GitHub Actions)
- [TODO] Deploy to Railway (backend) + Vercel (frontend)

Medium Priority

- [TODO] Add Redis caching layer (currently in-memory dict caches)
- [TODO] Implement monthly re-assessment flow
- [TODO] Add PYQs + Notes section (curated static files)
- [TODO] Implement industry trends section (Tavily weekly search)
- [TODO] Add progress tracking with streak calendar
- [TODO] Build mind games / engagement features
- [TODO] Consolidate two frontend codebases (root src/ vs frontend/)
- [TODO] Add dark mode support
- [TODO] Implement notifications for opportunity deadlines

Low Priority / Nice-to-Have

- [TODO] Add rate limiting on API endpoints
- [TODO] Implement user analytics dashboard
- [TODO] Add A/B testing for quiz difficulty distributions
- [TODO] Integrate Pinecone as alternative vector DB
- [TODO] Add WebSocket for real-time chatbot
- [TODO] Implement email notifications for weekly summaries
- [TODO] Add multi-language support (Hindi, regional languages)
- [TODO] Mobile app (React Native / Flutter)

10. Setup & Running Instructions

Prerequisites

- Python 3.10+
- Node.js 18+ and npm
- Google Gemini API key
- Firebase project (for auth + Firestore)

Backend Setup

```
cd backend
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt

# Create .env file with:
# GEMINI_API_KEY=your_key_here
# FIREBASE_PROJECT_ID=your_project_id

python main.py
# Server starts at http://localhost:8000
```

Frontend Setup

```
cd frontend
npm install

# Create .env file with Firebase config:
# VITE_FIREBASE_API_KEY=...
# VITE_FIREBASE_AUTH_DOMAIN=...
# VITE_FIREBASE_PROJECT_ID=...
# VITE_FIREBASE_STORAGE_BUCKET=...
# VITE_FIREBASE_MESSAGING_SENDER_ID=...
# VITE_FIREBASE_APP_ID=...

npm run dev
# Frontend starts at http://localhost:5173
```

Environment Variables

Variable	Description
GEMINI_API_KEY	Google Gemini API key (required for AI agents)
FIREBASE_PROJECT_ID	Firebase project ID (optional, defaults to ai-career-14
FIREBASE_SERVICE_ACCOUNT_JSON	Path to Firebase service account JSON (optional)
YOUTUBE_API_KEY	YouTube Data API key (optional, for course search)
SERPER_API_KEY	Serper.dev API key (optional, for opportunity search)
PORT	Backend port (default: 8000)
HOST	Backend host (default: 0.0.0.0)